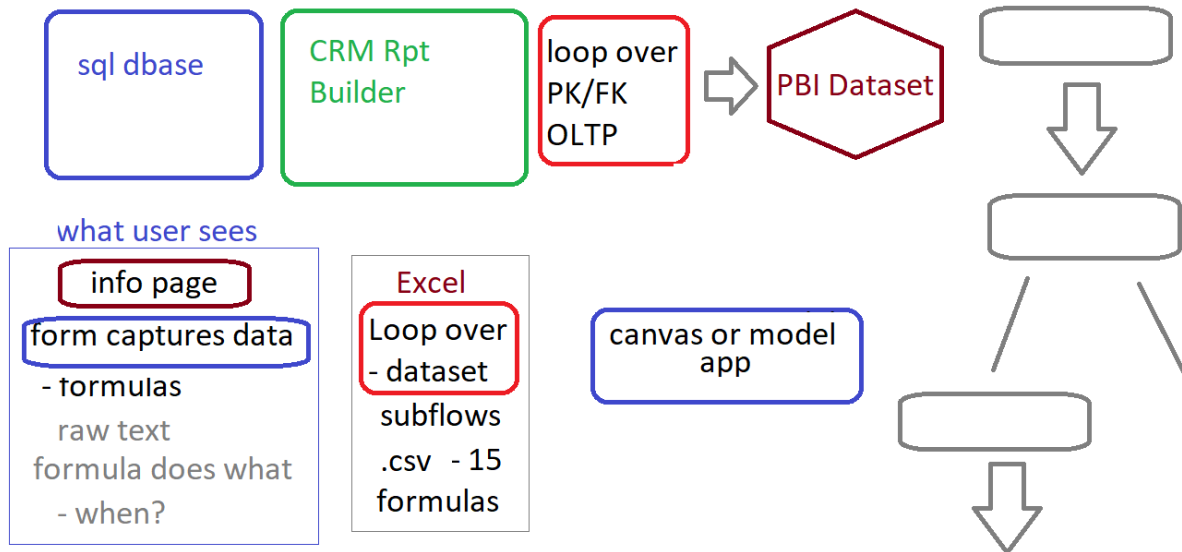

PA Subflow Loops

Contents

I have flow	2
Replace the "email Excel attachment Copilot	3
Option 2: Model-Driven App	6
Option 3: Keep Excel but Use App Launcher	7
Option 4: Power BI Writeback (My Favorite).....	8
You're thinking in exactly the direction I'd recommend.....	10
Replacing Excel Formulas.....	12
Where I Would Store the Data	13
Hybrid Approach.....	14
Best Option (#2)	18
Best Option (#3)	18
Best Option (#4)	19
Common Bottlenecks	22
Different ways to trigger the flow from the model-driven app.....	28

I have flow

- source PBI dataset to excel w/formulas - sent as email attachment to users w ms entra id to fill out and email back. How can I do this with either model or canvas app?



Are the Excel formulas business logic or just calculations?
If they're calculations, Power Fx can usually replace them.

Current State	Future State
1 PBI Dataset	1 PBI Dataset
2 ↓	2 ↓
3 Power Automate	3 Flow
4 ↓	4 ↓
5 Excel Template	5 Dataverse
6 ↓	6 ↓
7 Excel Formulas Calculate Values	7 Canvas App
	8 Display source data
	9 Perform calculations (Power Fx)
	10 Collect user inputs
	11 Submit
	12 ↓
	13 Dataverse
	14 ↓
	15 Power Automate
	16 ↓
	17 Downstream Processing

Replace the "email Excel attachment Copilot

Yes, you can absolutely replace the "email Excel attachment → user edits → emails back" pattern with either a **Canvas App** or a **Model-Driven App**, but I'd strongly lean toward **Canvas App** for your scenario.

Current Process

- 1 Power BI Dataset
- 2 ↓
- 3 Power Automate
- 4 ↓
- 5 Excel File w/ Formulas
- 6 ↓
- 7 Email to User
- 8 ↓
- 9 User Completes
- 10 ↓
- 11 Emails Back
- 12 ↓
- 13 Manual Processing

This process has a few challenges:

- Version control issues
- Email attachments everywhere
- Users can break formulas
- Difficult to track completion
- Hard to enforce data validation

Option 1 (Recommended): Canvas App

Instead of generating Excel files, write the Power BI dataset data into a table and let users update it through a Power App.

Architecture

- 1 Power BI Dataset
- 2 ↓
- 3 Power Automate
- 4 ↓
- 5 Dataverse Table
- 6 ↓
- 7 Canvas App
- 8 ↓
- 9 User Updates Data
- 10 ↓
- 11 Dataverse
- 12 ↓
- 13 Approval / Reporting Flow

How It Works

1. Create Dataverse Table

Example:

Column	Type
RequestID	Text
UserUPN	Text
SalesAmount	Currency
Forecast	Number
Comments	Multiline Text
Status	Choice
DueDate	Date

2. Load Data

Power Automate:

- 1 Refresh Power BI Dataset
- 2 ↓
- 3 Run DAX Query
- 4 ↓
- 5 For Each Record
- 6 ↓
- 7 Upsert Dataverse Record

3. Canvas App Security

Filter records by Entra ID:

- 1 Filter(
2 ForecastTable,
3 UserUPN = User().Email
4)

User automatically sees only their records.

No emails required.

4. Formula Replacement

Many Excel formulas can be recreated directly in Power Apps.

Example:

Excel:

```
1 =A1*B1
```

Canvas App:

```
1 txtQty.Value * txtPrice.Value
```

Or calculated columns in Dataverse.

5. Submit Changes

```
1 Patch(  
2   ForecastTable,  
3   ThisItem,  
4   {  
5     Forecast: Value(txtForecast.Text),  
6     Comments: txtComments.Text,  
7     Status: "Submitted"  
8   }  
9 )
```

Option 2: Model-Driven App

If the form is more data-entry oriented and less customized.

Architecture

1 Power BI Dataset

2 ↓

3 Dataverse

4 ↓

5 Model Driven App

Benefits:

- Faster to build
- Security built in
- Auditing
- Business Rules
- Approval workflows

Great when:

- Large user population
- Lots of fields
- Simple forms
- Manager approvals

Not as pretty as Canvas Apps.

Option 3: Keep Excel but Use App Launcher

If users absolutely need Excel formulas.

Flow

1 Power BI Dataset

2 ↓

3 Create Excel in SharePoint

4 ↓

5 Assign User

- 6 ↓
- 7 Canvas App displays link
- 8 ↓
- 9 User opens Excel Online
- 10 ↓
- 11 Submit

Instead of emailing files around, each user gets their own workbook stored in SharePoint.
This is a common "least disruption" upgrade.

Option 4: Power BI Writeback (My Favorite)

If this is truly a forecasting, planning, budgeting, or commentary process:

- 1 Power BI Dataset
- 2 ↓
- 3 Canvas App
- 4 ↓
- 5 Dataverse
- 6 ↓
- 7 Power BI Report

Embed the Canvas App directly inside Power BI.

Users:

1. Open report
2. See their numbers
3. Update forecast/comments
4. Save

5. Refresh report

No Excel at all.

If I Were Building This Today

I'd do:

- 1 Power BI Dataset
- 2 ↓
- 3 Power Automate
- 4 ↓
- 5 Dataverse
- 6 ↓
- 7 Canvas App
- 8 ↓
- 9 Manager Approval Flow
- 10 ↓
- 11 Power BI Dashboard

Then use:

- Entra ID security
- Dataverse tables
- Canvas App forms
- Power Automate approvals
- Power BI reporting

You'll eliminate attachment emails entirely and gain real-time tracking of who has/hasn't submitted.

For your AZ-204 and Power Platform journey, this is also the most scalable enterprise pattern and aligns closely with how Microsoft recommends replacing Excel collection processes with Power Apps + Dataverse.

You're thinking in exactly the direction I'd recommend.

For a **real-world modernization example** that aligns with both **PL-500/PAD migration concepts** and **Applied Skills (AB410)**, I'd stop thinking of the Excel workbook as the application and start thinking of it as a temporary rules engine.

Current State

- 1 PBI Dataset
- 2 ↓
- 3 Power Automate
- 4 ↓
- 5 Excel Template
- 6 ↓
- 7 Excel Formulas Calculate Values
- 8 ↓
- 9 User Enters Data
- 10 ↓
- 11 Email Back
- 12 ↓
- 13 Flow Processes Results

The key question is:

Are the Excel formulas business logic or just calculations?

If they're calculations, Power Fx can usually replace them.

Future State

- 1 PBI Dataset
- 2 ↓

3 Flow

4 ↓

5 Dataverse

6 ↓

7 Canvas App

8 |— Display source data

9 |— Perform calculations (Power Fx)

10 |— Collect user inputs

11 |— Submit

12 ↓

13 Dataverse

14 ↓

15 Power Automate

16 ↓

17 Downstream Processing

User Experience

User signs in.

Because they're authenticated with Entra ID:

1 User().Email

the app automatically filters to their assigned records.

Example:

1 Filter(

2 ForecastRequests,

3 AssignedTo = User().Email

4)

They see:

- Original PBI numbers
- Existing metrics
- Calculated values
- Input fields
- Submit button

No attachments.

No email responses.

No version issues.

Replacing Excel Formulas

Most formulas migrate surprisingly well.

Excel

```
1 =A1*B1
```

Power Fx

```
1 Value(txtQty.Text) * Value(txtPrice.Text)
```

Excel

```
1 =IF(A1>1000,"High","Low")
```

Power Fx

```
1 If(  
2   Amount > 1000,  
3   "High",  
4   "Low"  
5 )
```

Excel

1 =SUM(A1:A10)

Power Fx

```
1 Sum(  
2   CollectionName,  
3   Amount  
4 )
```

Excel

1 =VLOOKUP(...)

Power Fx

```
1 LookUp(  
2   ReferenceTable,  
3   ID = SelectedID  
4 )
```

Where I Would Store the Data

For AB410 training purposes I'd use:

Dataverse

Table 1

1 RequestHeader

Contains:

- RequestID
- UserUPN
- Status

- CreatedDate

Table 2

1 RequestData

Contains:

- Metric
- OriginalValue
- UserValue
- CalculatedValue

This demonstrates:

- Security
- Relationships
- Model-driven compatibility
- Power Pages compatibility later

Hybrid Approach

If some formulas are monstrous:

- 1 Nested IFs
- 2 INDEX/MATCH
- 3 XLOOKUP
- 4 Array formulas
- 5 Financial formulas

don't rewrite immediately.

Instead:

- 1 Canvas App
- 2 ↓

3 Power Automate

4 ↓

5 Office Script

6 ↓

7 Excel Calculates

8 ↓

9 Return Results

This is a great interim migration pattern.

You can gradually move the formulas into Power Fx over time.

What I'd Build for an Enterprise Client

Step 1

Nightly flow:

1 PBI Dataset

2 ↓

3 Dataverse

Step 2

Canvas App:

1 User opens app

Shows:

- Assigned records
- Historical values
- Current calculations
- Editable fields

Step 3

Power Fx performs calculations live.

Example:

```
1 lblVariance.Text =  
2 Text(  
3   UserForecast - CurrentForecast,  
4   "$#,###"  
5 )
```

Step 4

Submit:

```
1 Patch(...)
```

Step 5

Power Automate triggers:

```
1 Status = Submitted
```

and performs whatever your current "email returned workbook" flow does today.

Why This Fits AB410

This demonstrates nearly every modernization concept Microsoft wants people thinking about:

- Replace file-centric automation with app-centric automation
- Move business rules from spreadsheets into Power Fx
- Use Dataverse as the system of record
- Use Entra ID for security
- Use Power Automate only for orchestration
- Eliminate human dependency on email attachments

So my recommendation is:

Keep the Power BI dataset as the source, move the workbook data into Dataverse, recreate the Excel formulas in Power Fx where practical, and let the Canvas App become the user-facing form. The downstream flow can continue almost unchanged,

except it triggers from Dataverse updates instead of receiving returned Excel files. This is both a strong modernization story and a solid live demo architecture.

If you're talking about **Power BI Dataset (Semantic Model) → CRM-style reporting → SSRS/Paginated Report with subreports inside a row loop**, then the fastest path is usually **not** to feed the subreports directly from the semantic model.

The Performance Problem

A subreport inside a tablix/group executes repeatedly.

- 1 Main Report
- 2 Customer A
- 3 Subreport
- 4 Customer B
- 5 Subreport
- 6 Customer C
- 7 Subreport

If you have 1,000 customers, the subreport can be processed 1,000 times. Microsoft specifically notes that subreports in data regions are processed for every row and can significantly impact performance. Nested data regions often perform better than subreports. [\[learn.microsoft.com\]](https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-subreports), [\[learn.microsoft.com\]](https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-subreports)

Best Option (#1)

Replace:

- 1 Main Report
- 2 └ Subreport Loop

with:

- 1 Single DAX Query
- 2 ↓
- 3 One Dataset
- 4 ↓

5 Grouped Tablix

6 ↓

7 Nested Regions

Microsoft recommends using nested data regions instead of subreports when possible because they are processed more efficiently. [\[learn.microsoft.com\]](https://learn.microsoft.com), [\[learn.microsoft.com\]](https://learn.microsoft.com)

Best Option (#2)

If the source is a Power BI semantic model:

1 Fabric / Power BI Semantic Model

2 ↓

3 Paginated Report

4 ↓

5 DAX Query

Push calculations into:

- Semantic Model measures
- Calculated columns
- Power Query transformations

rather than report expressions. Microsoft notes that moving calculations into the dataset/model improves scalability and performance. [\[learn.microsoft.com\]](https://learn.microsoft.com)

Best Option (#3)

For CRM-style reporting with lots of detail sections:

Create a flattened reporting table.

1 FactCustomerActivity

2 -----

3 CustomerID

- 4 CustomerName
- 5 Opportunity
- 6 Revenue
- 7 Forecast
- 8 CaseCount

instead of:

- 1 Customer
- 2 ↓
- 3 Opportunity Subreport
- 4 ↓
- 5 Case Subreport
- 6 ↓
- 7 Forecast Subreport

A denormalized reporting model plus grouped tablix is almost always faster than iterative subreport execution.

Best Option (#4)

If you're already in Fabric/Power BI:

- 1 PBI Semantic Model
- 2 ↓
- 3 Paginated Report

Use:

- Import mode semantic model
- Star schema
- Aggregation tables
- Optimized DAX

Microsoft identifies semantic model design and storage mode as primary performance drivers. [\[learn.microsoft.com\]](https://learn.microsoft.com), [\[sqlbi.com\]](https://sqlbi.com)

What I'd Do for a Modern CRM Migration

Instead of:

- 1 CRM
- 2 ↓
- 3 SSRS
- 4 ↓
- 5 Subreports

I'd build:

- 1 CRM / Dataverse
- 2 ↓
- 3 Fabric Lakehouse
- 4 ↓
- 5 Semantic Model
- 6 ↓
- 7 Paginated Report
- 8 ↓
- 9 Single DAX Dataset

and completely eliminate the subreport loop.

For most CRM migrations I see, **removing subreports and replacing them with one semantic-model-driven dataset plus grouped tablix regions produces the biggest performance gain**, often much more than tuning DAX alone. [\[learn.microsoft.com\]](https://learn.microsoft.com), [\[learn.microsoft.com\]](https://learn.microsoft.com)

if subflows are about looping (through a table - row by row) it seems to act like a table scan in the length of time to run. What improves the wait times or changes sub behavior & what tool acts like an execution plan for query improvement?

A single transaction often beats thousands of transactions.

"Can this become a single query and a grouped result set?"

The biggest gains usually come from eliminating nested loops, just like replacing nested cursor logic in SQL with a set-based operation.

Good analogy. In **Power Automate Desktop (PAD)** and cloud flows, many subflows behave like an inefficient table scan when they're invoked inside a loop.

What's Actually Happening

Imagine:

- 1 For Each Row in Excel
- 2 Call Subflow
- 3 Next Row

If there are 5,000 rows:

- 1 5,000 loop iterations
- 2 5,000 subflow executions

The overhead of:

- Context switching
- Variable passing
- Excel I/O
- Dataverse I/O
- API calls
- Browser actions

often dominates the actual work.

Common Bottlenecks

1. Excel Access

Worst pattern:

- 1 Loop
- 2 Read Excel Cell
- 3 Loop

Better:

- 1 Read Entire Worksheet
- 2 ↓
- 3 DataTable
- 4 ↓
- 5 Loop Memory Object

Excel open/read/write operations are among the slowest desktop actions.

2. Database Chattiness

Worst:

- 1 Loop
- 2 SELECT
- 3 Loop

Better:

- 1 SELECT *
- 2 FROM Customers
- 3 WHERE Status='Active'

once, then process locally.

Same principle as SQL optimization.

3. Subflow Call Overhead

Every subflow call has startup cost.

Bad:

- 1 Loop
- 2 Call Subflow
- 3 Loop

Better:

- 1 Pass entire DataTable
- to one subflow.

4. UI Automation

Worst:

- 1 Click
- 2 Wait
- 3 Read
- 4 Click
- 5 Wait
- 6 Read

Browser and desktop UI actions are typically 10x to 100x slower than API-based operations.

AB410 and PL-500 modernization guidance generally pushes toward APIs and connectors rather than UI automation whenever possible.

What Changes Subflow Behavior?

Chunking

Instead of:

1 5000 rows

process:

1 500 row batches

or

1 1000 row batches

This reduces memory pressure and improves recovery.

Filter Before Loop

Instead of:

1 For Every Row

2 If Status = Active

do:

1 Get Active Rows

2 ↓

3 Loop

Same concept as SQL predicate pushdown.

Replace Loops with Bulk Operations

Example:

1 Apply to Each

2 Create Record

versus

1 Bulk Create Records

A single transaction often beats thousands of transactions.

Closest Thing to an Execution Plan

Power Automate doesn't have a true SQL-style execution plan.

However, these are the closest equivalents.

Cloud Flow

Flow Analytics

1 Flow

2 ↓

3 Analytics

Look at:

- Action duration
- Success rate
- Throughput
- Bottlenecks

Run History

This is your actual execution plan.

Expand:

1 Apply to Each

and look for:

1 Action = 50 ms

2 Action = 8 sec

3 Action = 12 sec

You'll normally discover one connector causing 90% of runtime.

Compose Timers

A trick I use:

1 Compose StartTime

before an action.

1 Compose EndTime

after it.

Calculate elapsed times and identify hot spots.

Power BI Equivalent

For semantic models, the "execution plan" tools are:

Performance Analyzer

Inside Power BI Desktop.

Shows:

- Visual query time
- DAX execution time
- Rendering time

DAX Studio

This is probably the nearest equivalent to SQL Server execution plans.

You can see:

- Query plans
- Storage Engine activity
- Formula Engine activity
- Server timings

SQLBI specifically recommends DAX Studio when analyzing semantic model performance. [\[sqlbi.com\]](https://sqlbi.com)

VertiPaq Analyzer

Shows:

- Cardinality
- Memory usage
- Table size
- Compression

Useful for model tuning. [\[sqlbi.com\]](https://sqlbi.com)

My Rule of Thumb

When I see:

- 1 Loop
- 2 Subflow
- 3 Loop
- 4 Query

I immediately think:

- "Can this become a single query and a grouped result set?"
- The biggest gains usually come from eliminating nested loops, just like replacing nested cursor logic in SQL with a set-based operation.

For someone coming from a database tuning mindset, think of PAD and Power Automate optimization as:

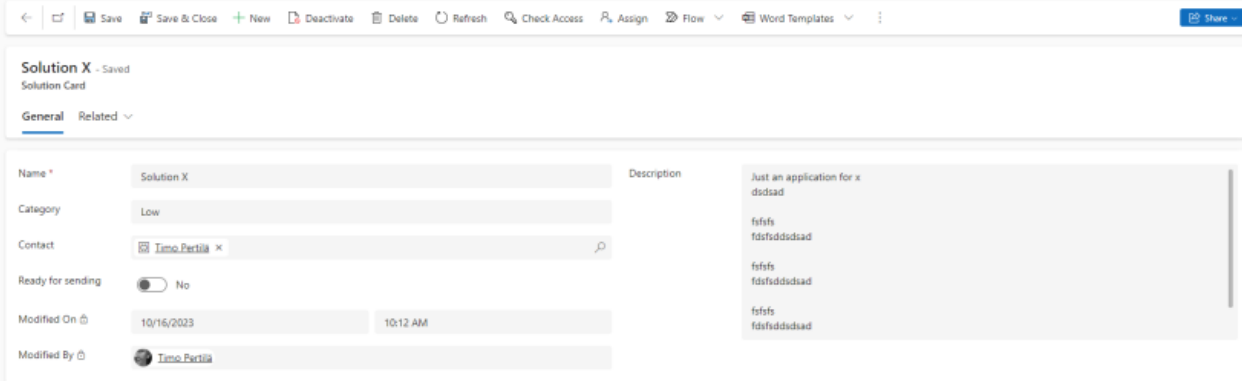
- 1 Filter early
- 2 Reduce I/O
- 3 Avoid nested loops (subflows?)
- 4 Batch updates
- 5 Cache data in memory
- 6 Prefer APIs over UI automation

Those five principles typically cut run times more than any individual PAD setting or subflow tweak.

Different ways to trigger the flow from the model-driven app

Written by: [Timo Pertilä](#)

Posted on: 2024-11-12



The screenshot shows a 'Solution Card' form in the Power Platform. The title bar includes navigation icons and a 'Share' button. The form is titled 'Solution X - Saved' and 'Solution Card'. It has two tabs: 'General' (selected) and 'Related'. The 'General' tab contains the following fields:

- Name ***: Solution X
- Category**: Low
- Contact**: Timo Pertilä (with a search icon)
- Ready for sending**: No (toggle switch)
- Modified On**: 10/16/2023 10:12 AM
- Modified By**: Timo Pertilä
- Description**: Just an application for x ddsad
fefs
fdsfdddsad
fefs
fdsfdddsad
fefs
fdsfdddsad

Model-driven applications may seem painfully limited if you've learned Power Platform by making Canvas applications.

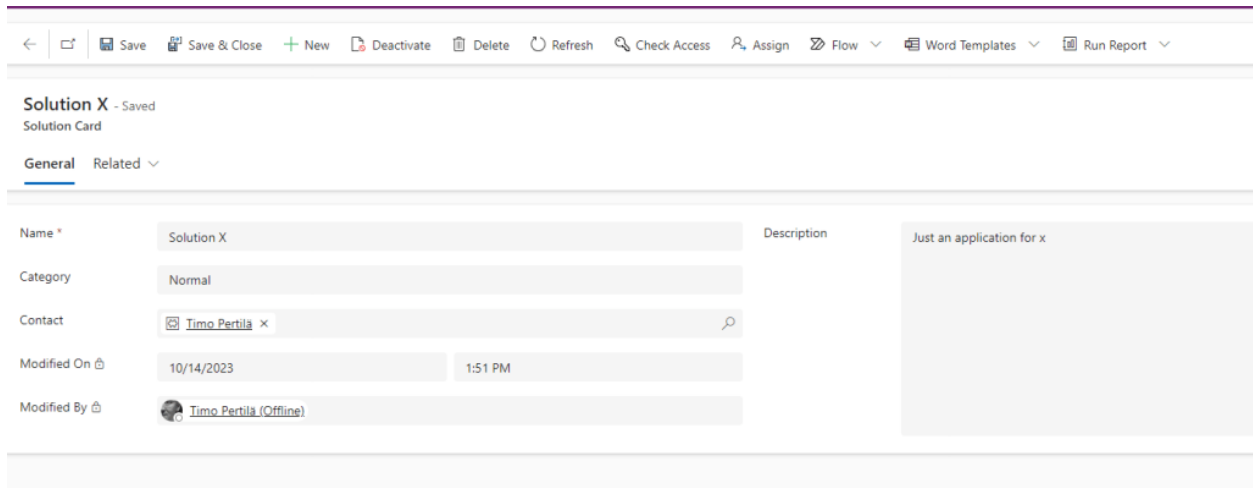
And they are. That's the whole idea and beauty of them. Model-driven apps are based on a data model. Tuning the UI is difficult or impossible. On the other hand, model-driven apps are reliable. When working with them, you don't have to wonder and test whether saving the form works. It just works.

Sometimes, however, there is a need to enrich the functionalities of these wonderful model-driven apps. This time we will get to know one such need.

That is, how do you implement the trigger function in the model-driven app form?

Example – Solution Cards

Let's consider a simple application that maintains information about the applications used by the organization.



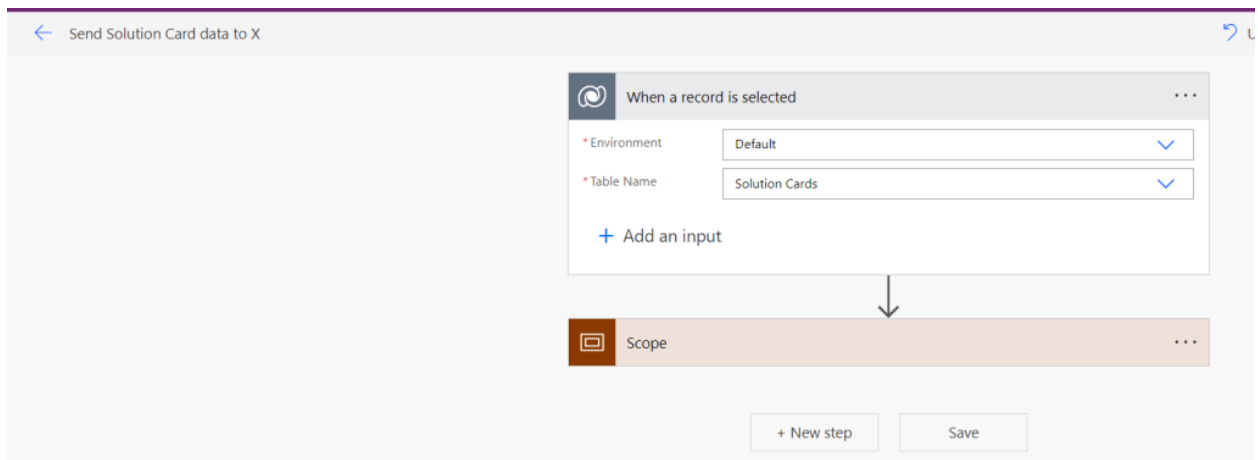
Once the application information is entered, it is sent to system x. When will the data be ready to be sent? It is decided by the user.

The user must therefore be offered a way to initiate data transmission.

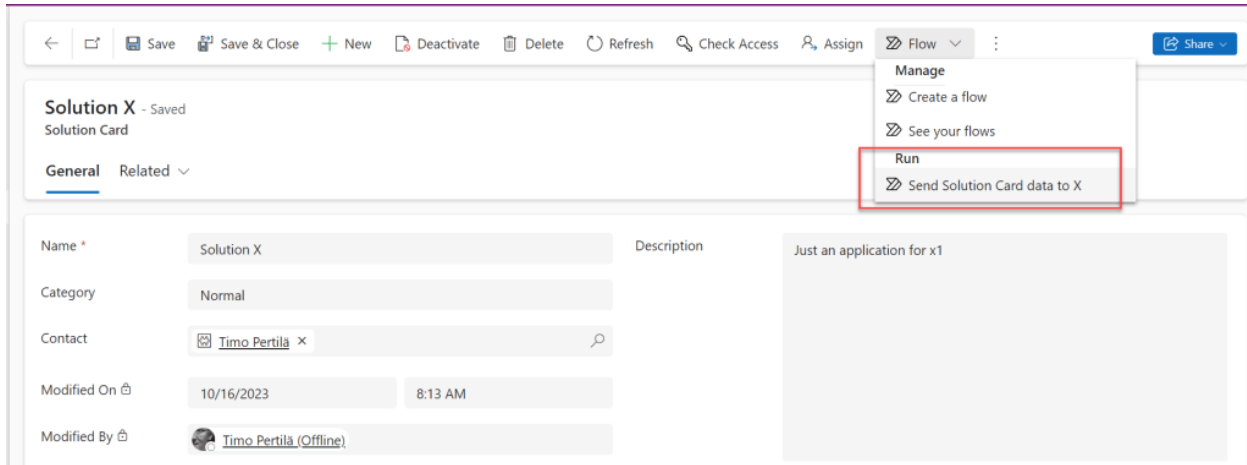
There are several implementation options.

Starting Flow from the form

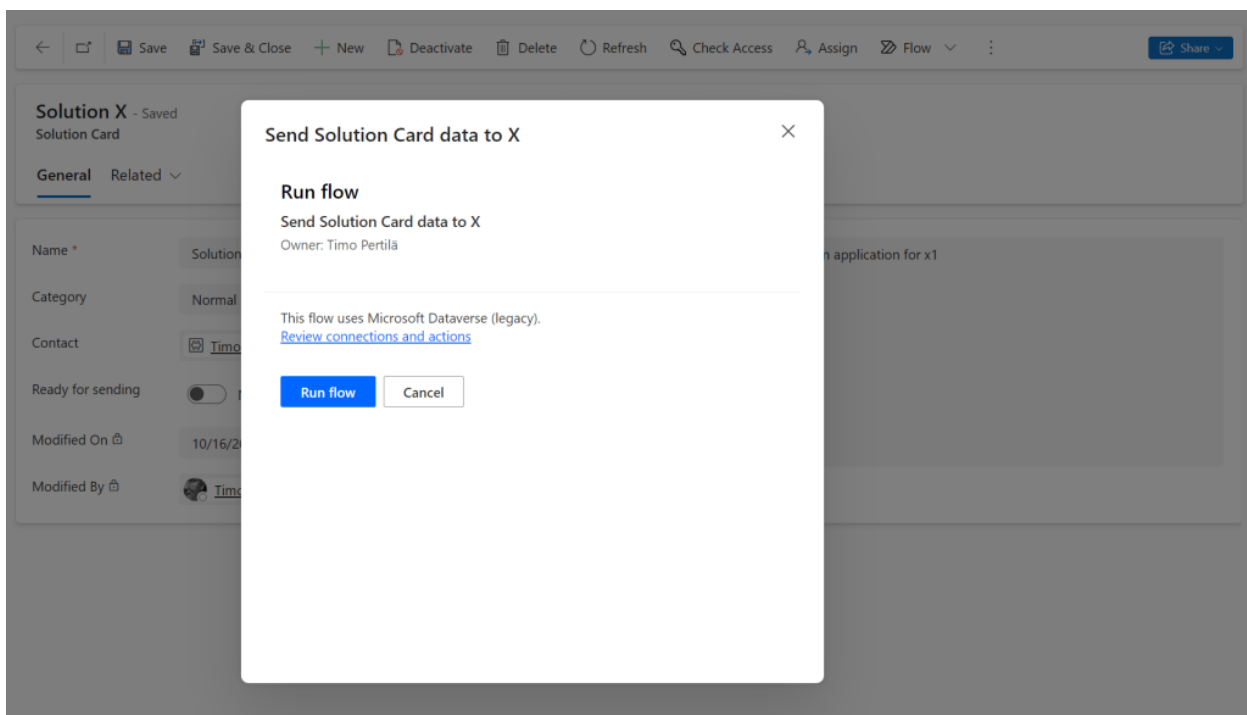
Let's start with the simplest way. A flow is created and it is triggered from the form (When a record is selected). Flow forwards to the system x the information of the selected solution card.



Now the user can forward the solution information from the Flow menu.



Easy, but the user experience is not very special. And you can't control that.



The user also does not receive any feedback as to whether the data transmission was successful. Of course, Flow can send a team message to the user, for example, if the transmission goes wrong. But it's a little bit workaround I think.

Toggle

The traditional way to solve this type of problem is to add a yes/no column to the solution card. The value of the field indicates whether the information in the form is ready to be forwarded.

New column

Add existing column

Advanced

Button on MDA

Tables

Solution Card

Columns

Display name

Name

Category	tp_Category
Contact	tp_Contact
Created By	CreatedBy
Created By (Delegate)	CreatedOnBehalfBy
Created On	CreatedOn
Description	tp_Description
Import Sequence Number	ImportSequenceNumber
Modified By	ModifiedBy
Modified By (Delegate)	ModifiedOnBehalfBy
Modified On	ModifiedOn
Name Primary name column	tp_Name
Owner	OwnerId
Owning Business Unit	OwningBusinessUnit
Owning Team	OwningTeam
Owning User	OwningUser
Record Created On	OverriddenCreatedOn
Solution Card	tp_SolutionCardId
Status	statecode

New column

Previously called fields. [Learn more](#)

Display name

Ready for sending

Description

Data type

Yes/no

Behavior

Simple

Required

Optional

Searchable

Choices

Label	Value
No	0
Yes	1

Default choice

No

Advanced options

Save

Cancel

Let's add a field to the form and change it to a toggle.

Power Apps | Form

Enactment
Timo's Dev Environment

Ready for sending
Yes/No

Properties Events Business rules

Table column

Ready for sending ⓘ

Edit table column

Label *

Ready for sending

☐ Hide label

☐ Hide on phone ⓘ

☐ Hide ⓘ

☐ Lock ⓘ

☐ Read-only

Formatting

Form field width

1 column

Components

+ Component 2

Back Undo Redo Add Delete Form field Component Form libraries Business rules Form properties Form settings ... Save a copy Save and publish

Table columns

Search

+ New table column

☒ Show only unused table columns

Created By

Created By (Delegate)

Created On

Modified By (Delegate)

Owning Business Unit

Status

Status Reason

New Solution Card

Solution Card

General Related

Name	Description
Category	
Contact	
Ready for sending	
Modified On	
Modified By	

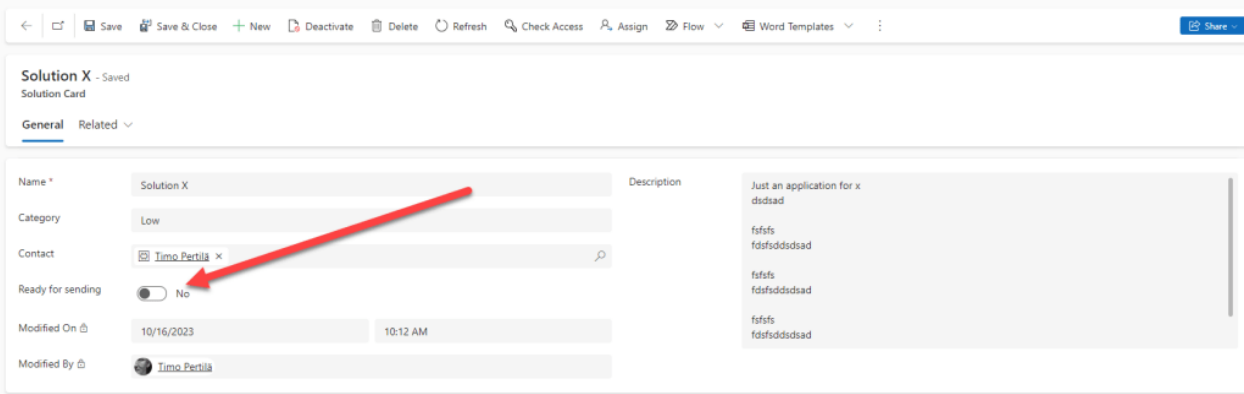
Add component

Search

☐ Toggle 3

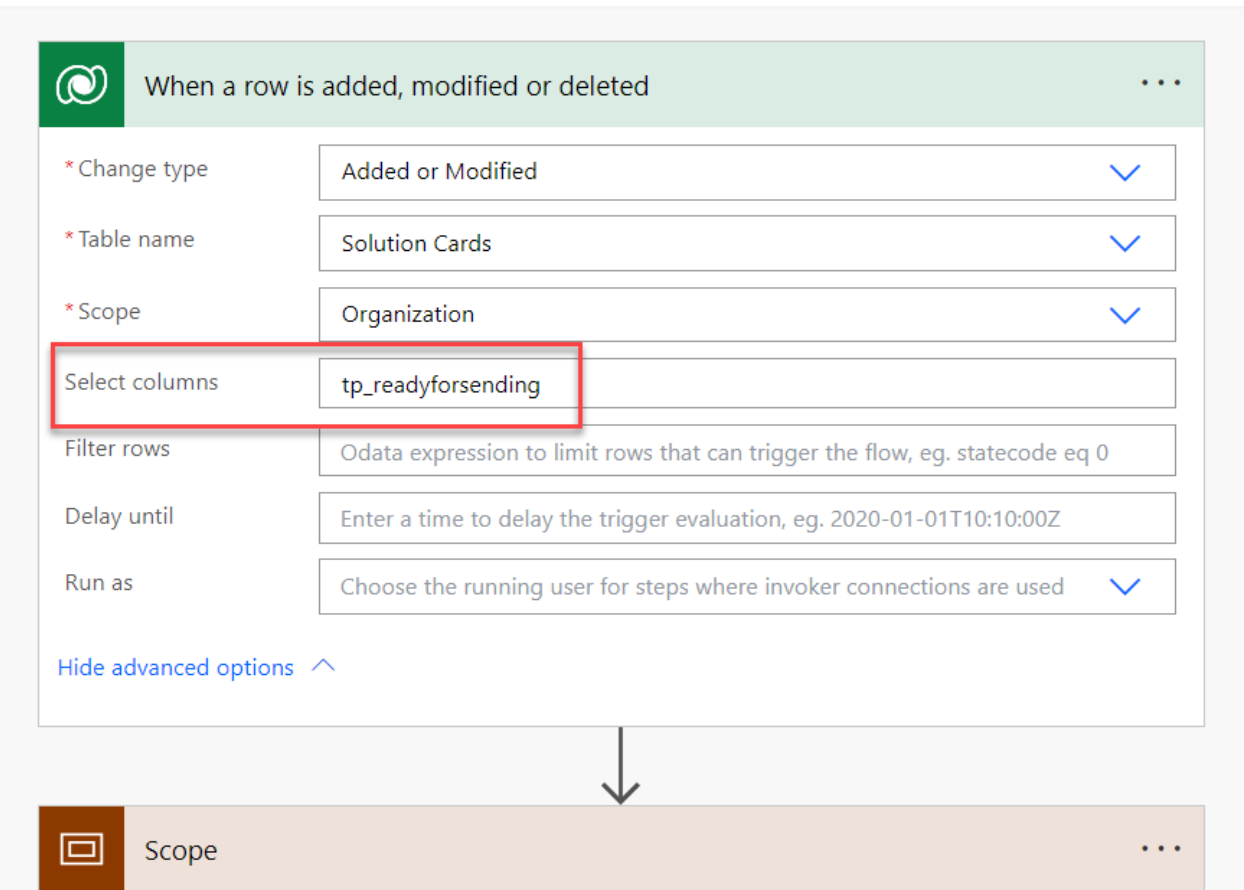
Canvas app

The user uses the toggle to mark the information in the form to be ready for forwarding.



The screenshot shows a form for 'Solution X'. The 'Ready for sending' toggle is set to 'No'. A red arrow points to this toggle. The form includes fields for Name, Category, Contact, Description, and Modified On/By.

Data is sent with a flow that starts when the value of the toggle (tp_readyforsending) changes.



The screenshot shows the configuration for a trigger 'When a row is added, modified or deleted'. The 'Select columns' field is highlighted with a red box and contains 'tp_readyforsending'. Other fields include 'Change type' (Added or Modified), 'Table name' (Solution Cards), 'Scope' (Organization), 'Filter rows' (Odata expression to limit rows that can trigger the flow, eg. statecode eq 0), 'Delay until' (Enter a time to delay the trigger evaluation, eg. 2020-01-01T10:10:00Z), and 'Run as' (Choose the running user for steps where invoker connections are used).

Strictly speaking, flow starts only when the toggle changes to true.

Trigger Conditions: @equals(triggerOutputs()?['body/tp_readyforsending'], true)

Settings for 'When a row is added, modified or deleted'

Custom Tracking Id

Set the tracking id for the run. For split-on this tracking id is for the initiating request.

Tracking Id

Secure Inputs

Secure inputs of the operation.

Secure Inputs

☐ Off

Secure Outputs

Secure outputs of the operation and references of output properties.

Secure Outputs

☐ Off

Retry Policy

A retry policy applies to intermittent failures, characterized as HTTP status codes 408, 429, and 5xx, in addition to any connectivity exceptions. The default is an exponential interval policy set to retry 4 times.

Type

Concurrency Control

Limit number of concurrent runs of the flow, or leave it off to run as many as possible at the same time. Concurrency control changes the way new runs are queued. It cannot be undone once enabled.

Limit

☐ Off

Trigger Conditions

Specify one or more expressions which must be true for the trigger to fire.

After the information has been forwarded, we return the toggle to its original state.

↓

 Update a row ...

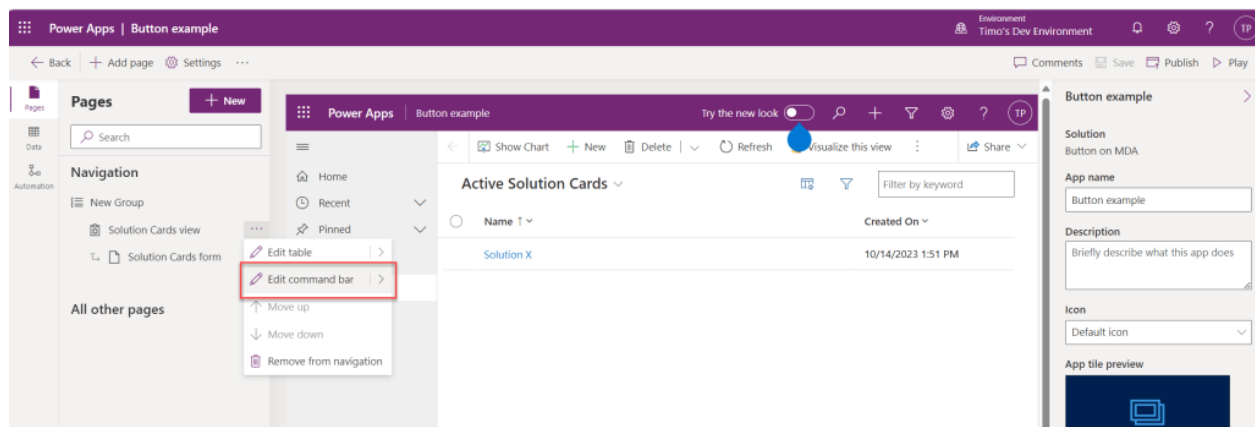
* Table name	Solution Cards ▼
* Row ID	 Solution Card ×
Name	
Category	▼
Contact (AAD Users)	
Description	
Owner (Owners)	Owner Id
Ready for sending	No ▼
Status	Status of the Solution Card ▼
Status Reason	Reason for the status of the Solution Card ▼
Time Zone Rule Version Number	For internal use only.
UTC Conversion Time Zone Code	Time zone code that was in use when the record was created.

[Hide advanced options](#) ^

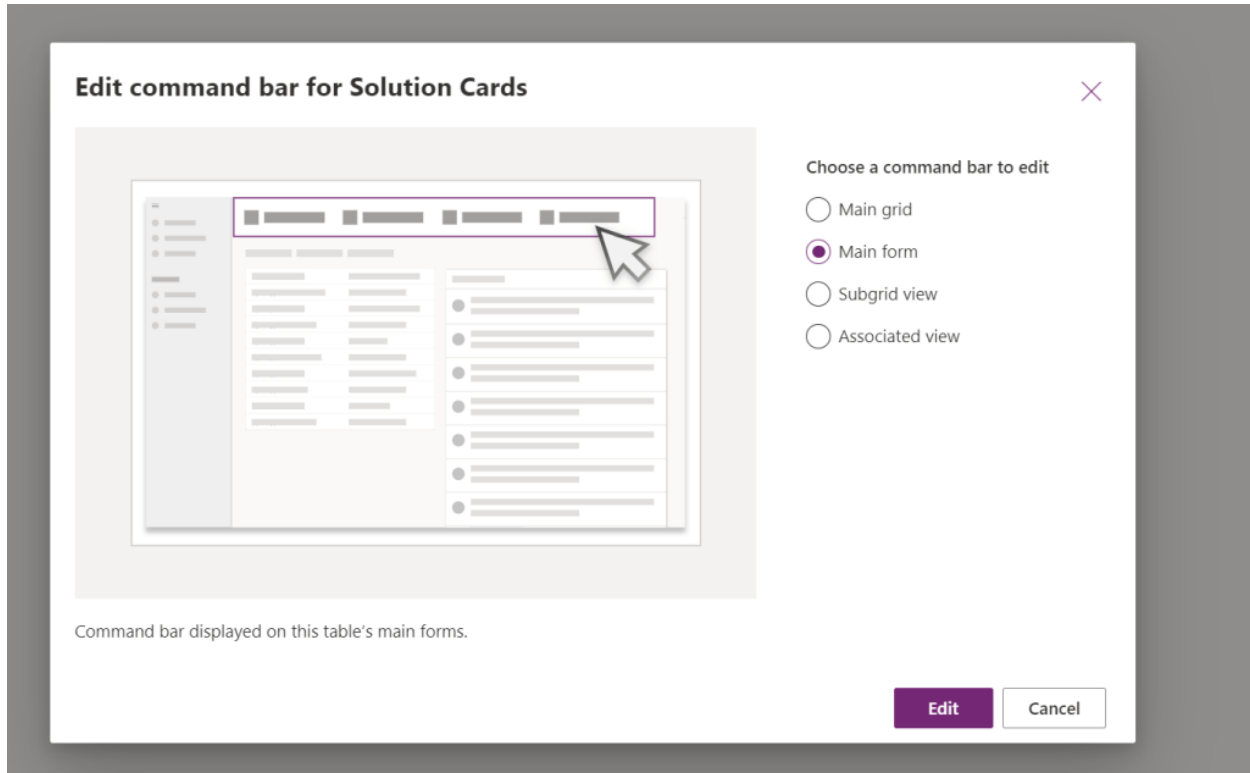
Works! But could this be done differently?

Marking the form as complete from the command bar

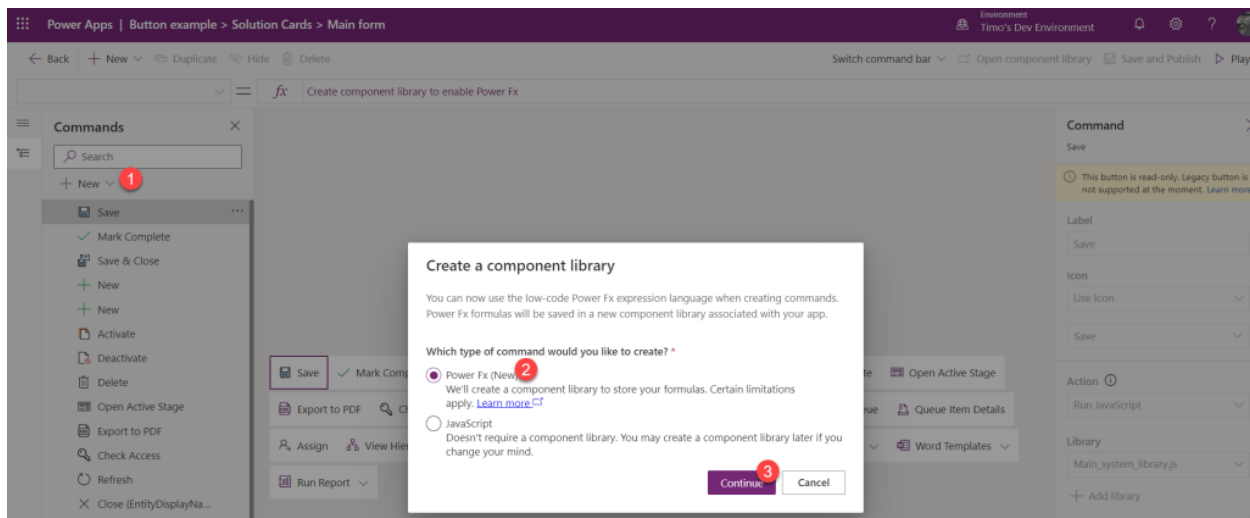
What if the toggle introduced above was used from the command bar? Let's edit the command bar of the solution card.



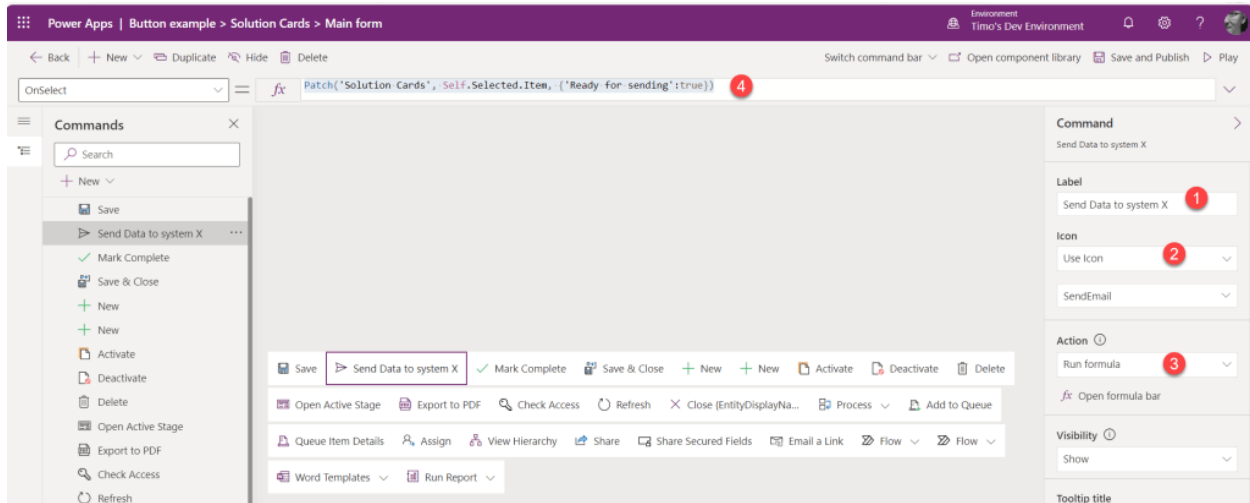
We want to edit the main form menu.



A PowerFx (2) button is added to the (1) menu.



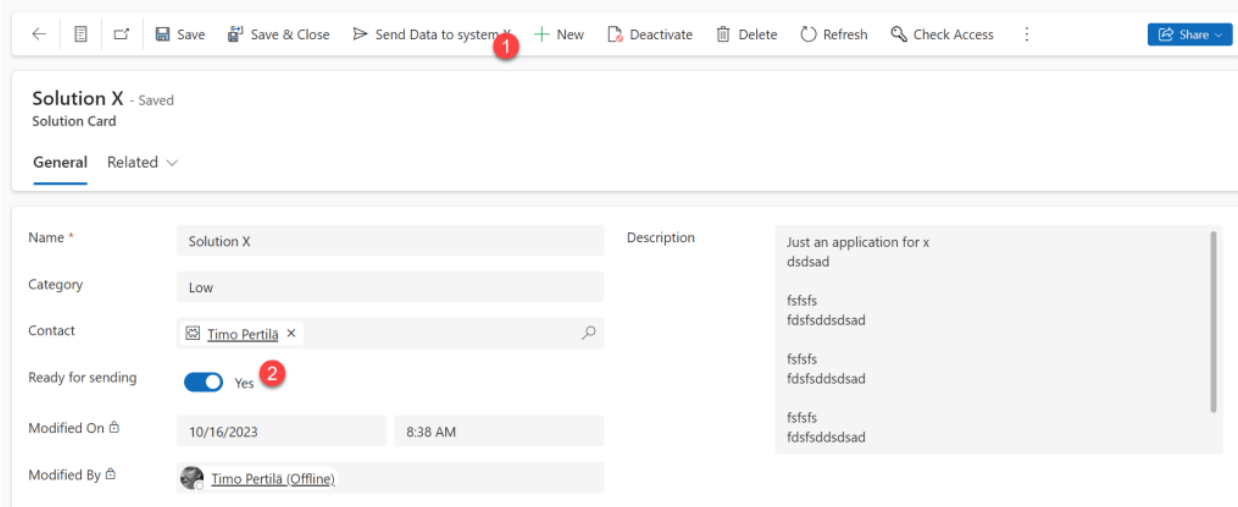
Let's name the button (1), change its icon (2) to a more appropriate one and define the button to execute the PowerFx command (3).



After this, the work is familiar from the canvas Power Apps world. When the button is pressed (OnSelect), the Ready for Sending column in the line is updated to true (4).

OnSelect: Patch('Solution Cards', Self.Selected.Item, {'Ready for sending':true})

When the user presses the button (1), the solution card is marked as ready for sending (2) and saved. If the user has unsaved changes on the form, they will be saved at the same time.

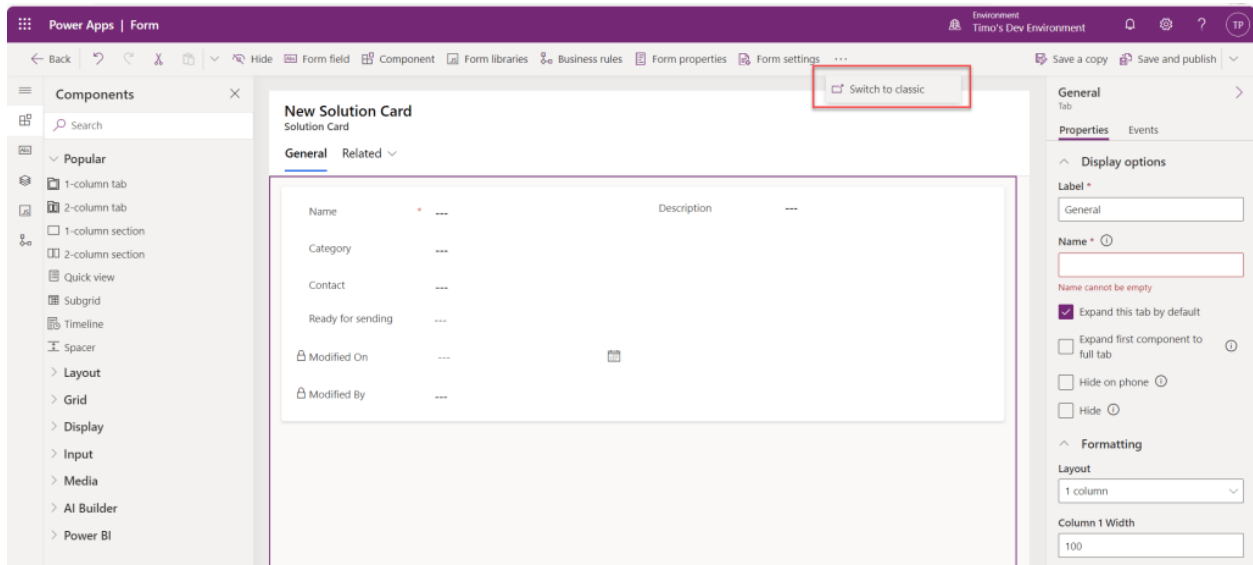


The toggle can be removed from the form.

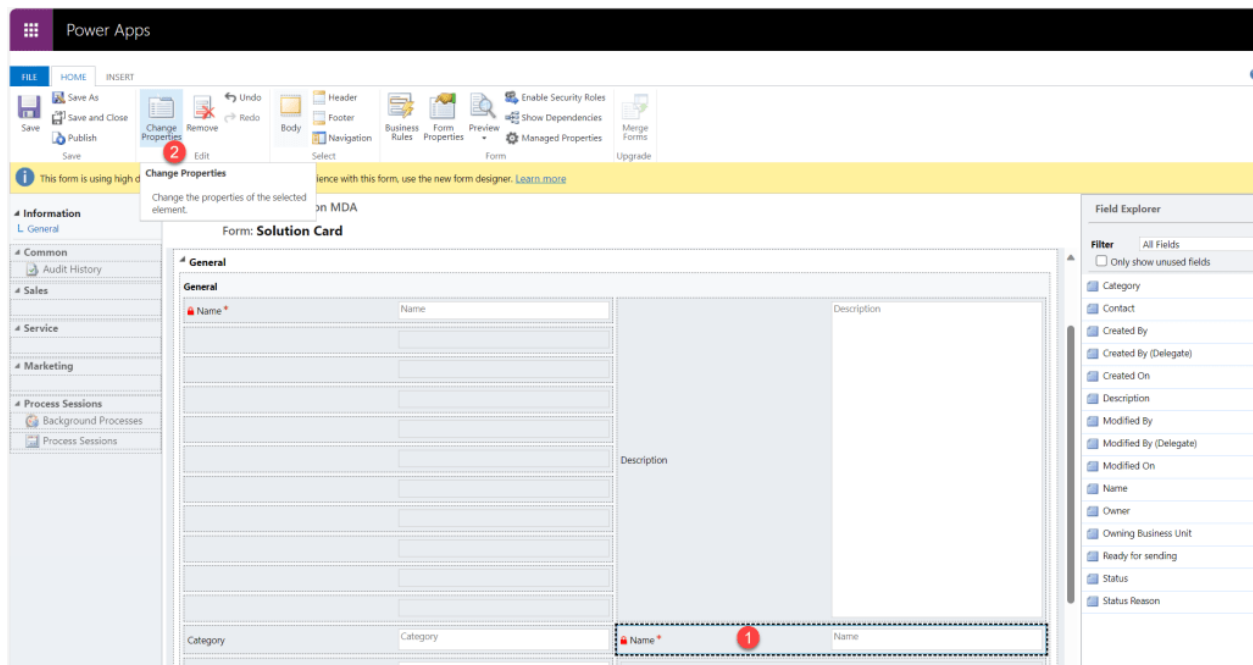
Embedding the Canvas application on a form

We would like to tell the user immediately after pressing the button whether the transmission was successful or not. This can be done by using the embedded canvas app.

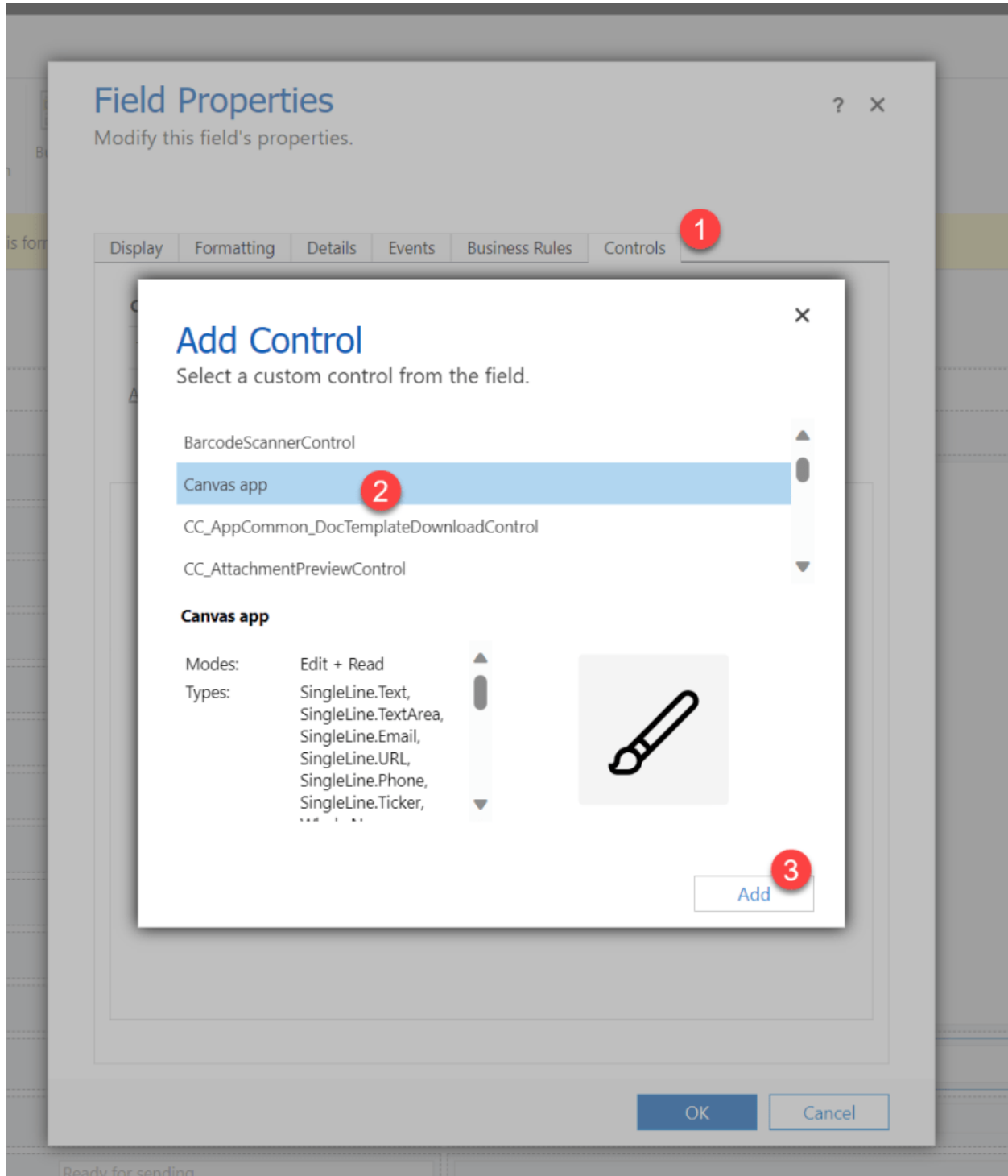
Let's add a simple canvas app to the form, with which the sending is done. Adding is easiest from the old user interface (classic).



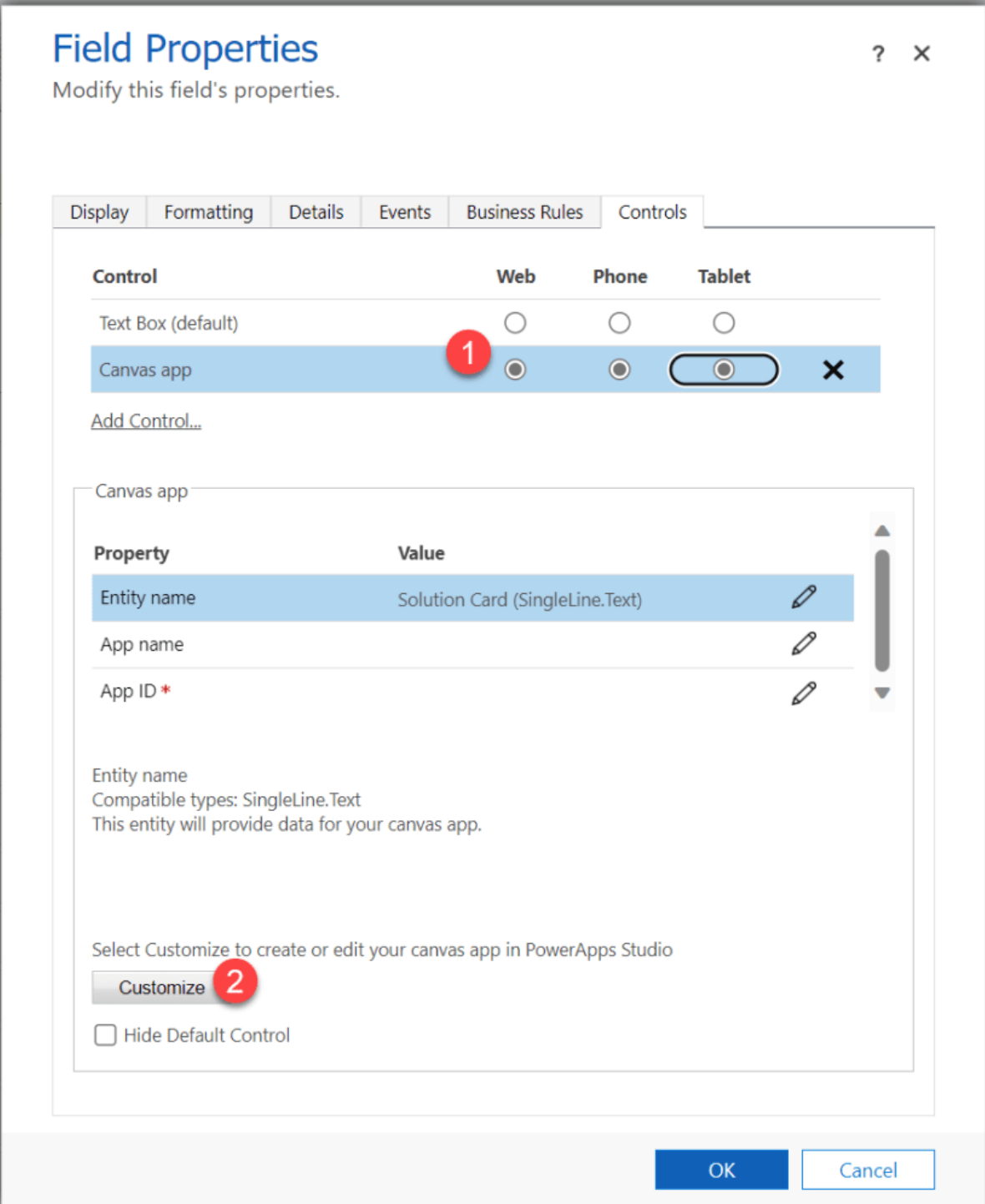
A field (1, e.g. Name) is added to the form and the properties of that field are modified (2, Change Properties).



Let's go to the Controls tab (1), add a control and select Canvas app (2).



Let's change the control to use the canvas application (1) and open it (2, Customize).



Field Properties ? ×

Modify this field's properties.

Display Formatting Details Events Business Rules Controls

Control	Web	Phone	Tablet
Text Box (default)	☐	☐	☐
Canvas app	☒	☒	☒

[Add Control...](#)

Canvas app

Property	Value
Entity name	Solution Card (SingleLine.Text)
App name	
App ID *	

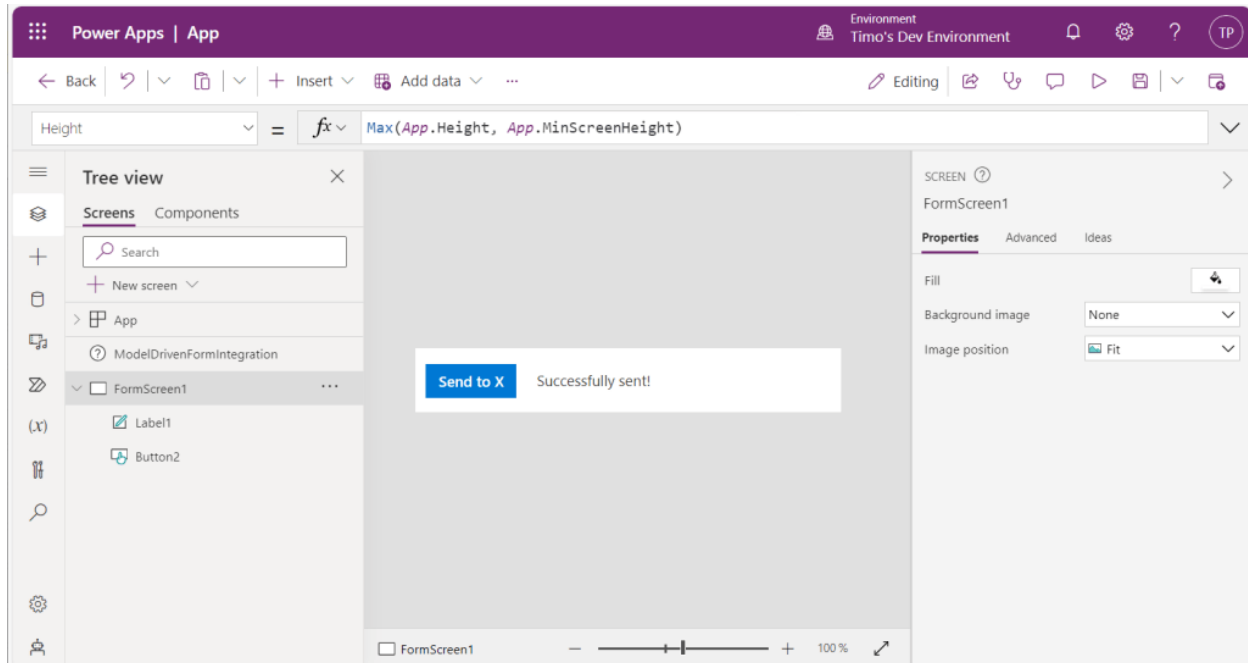
Entity name
Compatible types: SingleLine.Text
This entity will provide data for your canvas app.

Select Customize to create or edit your canvas app in PowerApps Studio

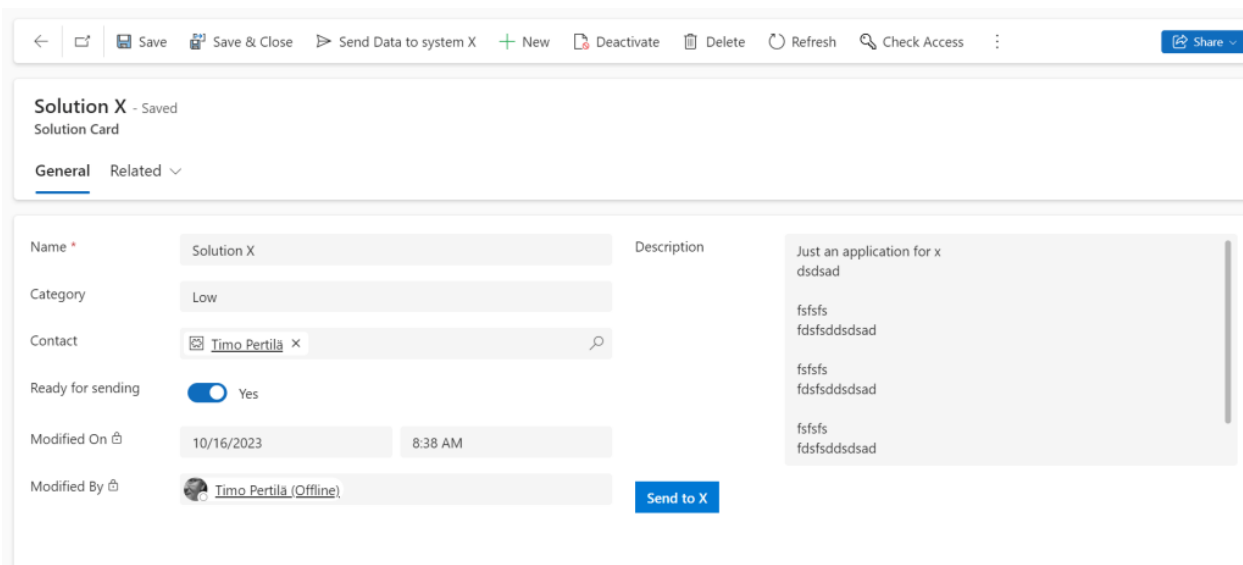
☒ Customize ☐ Hide Default Control

OK Cancel

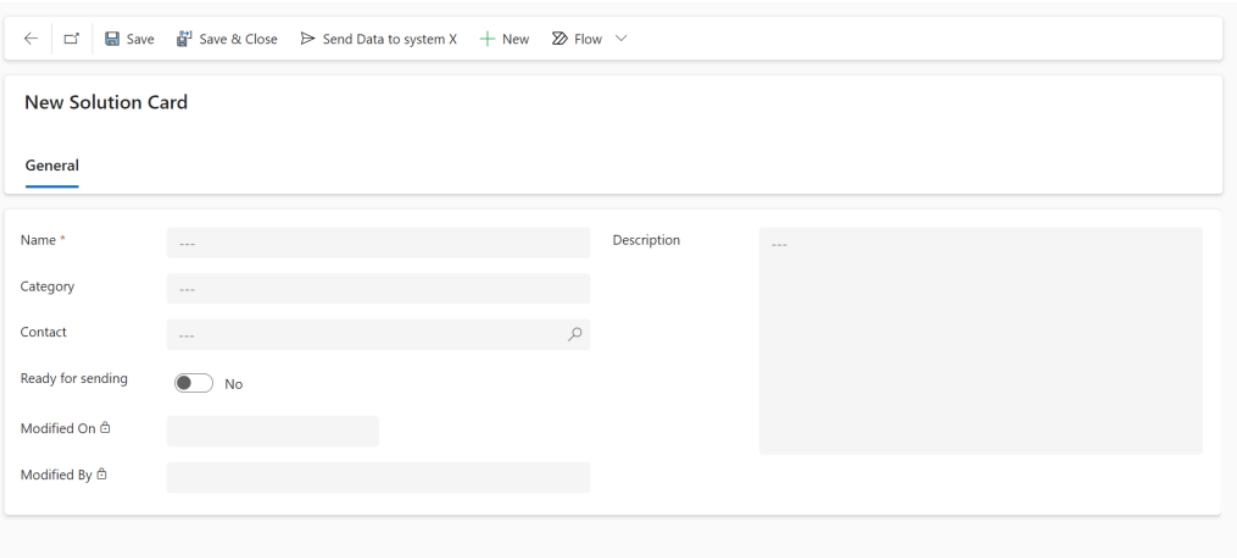
Then we build a responsive canvas app that forwards the form information. We can immediately show the user feedback on the success of the transmission.



This way we have a button embedded on a form in a model-driven app.



Note that the button is not visible when creating a new card. The canvas app embedded in the form is only visible when data is being edited (when it knows which Dataverse row it is related to).



← □ Save Save & Close Send Data to system X + New Flow

New Solution Card

General

Name * Description

Category

Contact

Ready for sending ☐ No

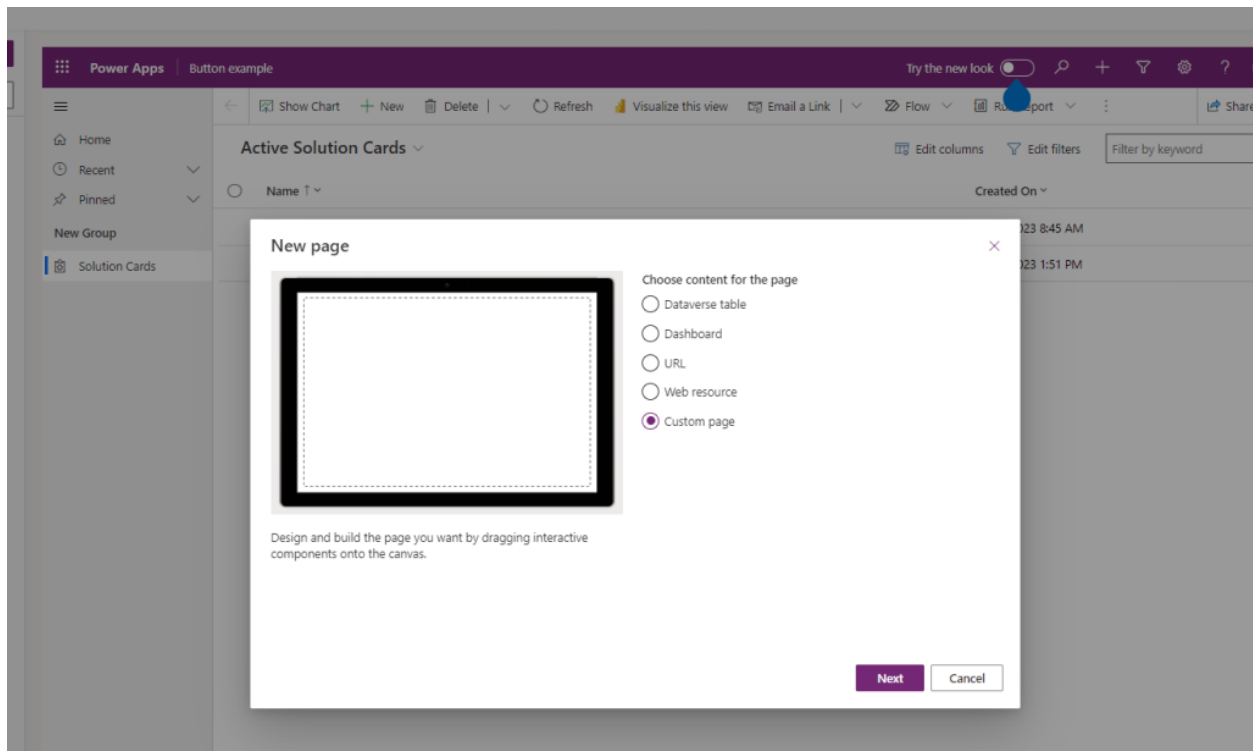
Modified On

Modified By

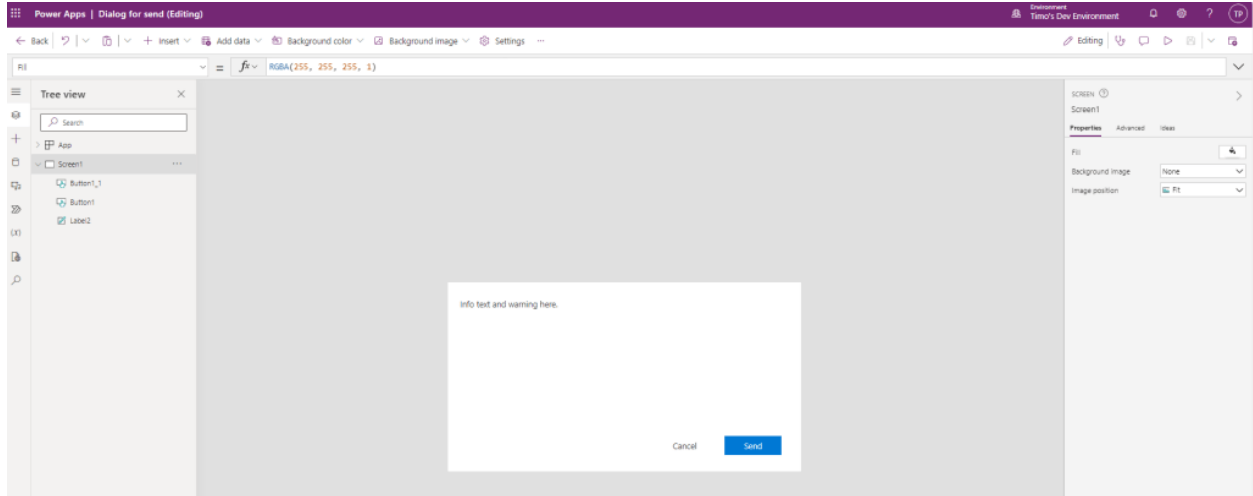
Opening a dialog from the command bar

We can open our own dialog from the command bar (custom Dialog). The dialog is practically a Power Apps (Custom page).

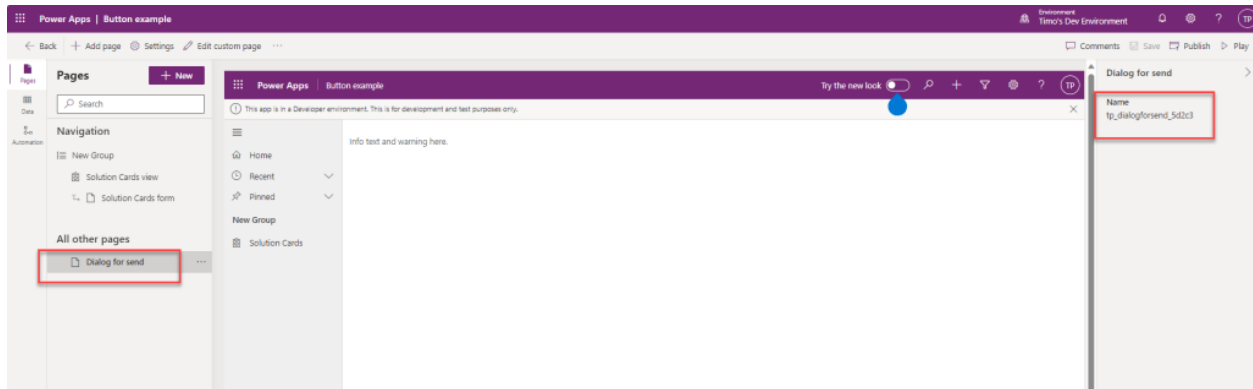
Let's create one.



Our simple dialog looks like this. This Power Apps performs the actual transmission and tells the user if it was successful.



Let's extract the name of the created dialog, we need it in the JavaScript that opens the dialog.



Then we create a javascript file.

```
function openCustomPageDialog(primaryControl, firstSelectedItemId,
selectedEntityTypename)
```

```
{
```

```
    // Centered Dialog
```

```
    var pageInput = {
```

```
        pageType: "custom",
```

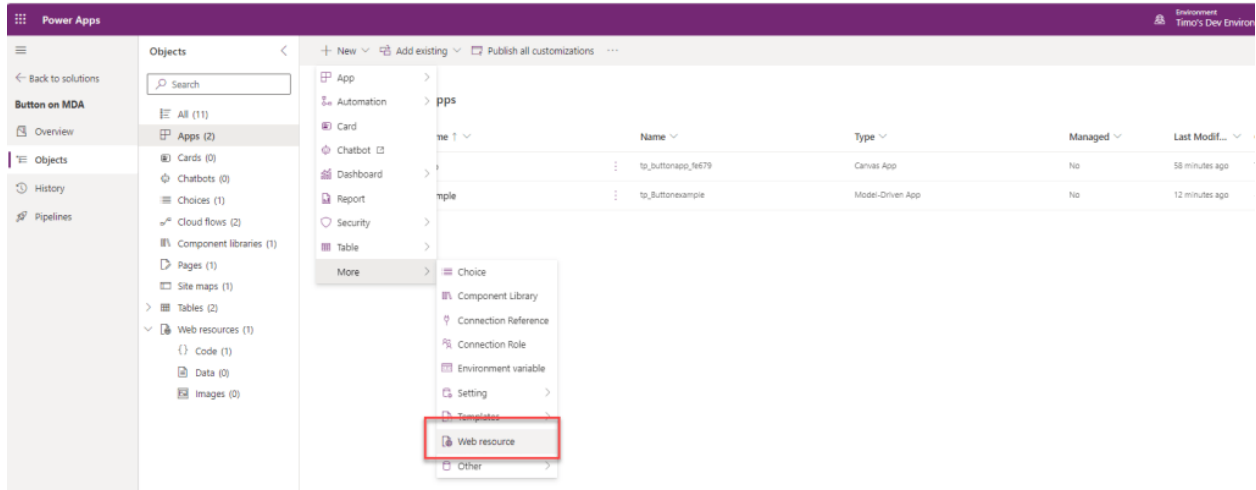
```
        name: "tp_dialogforsend_5d2c3",
```

```
        entityType: selectedEntityTypename,
```

```
        recordId: firstSelectedItemId
```

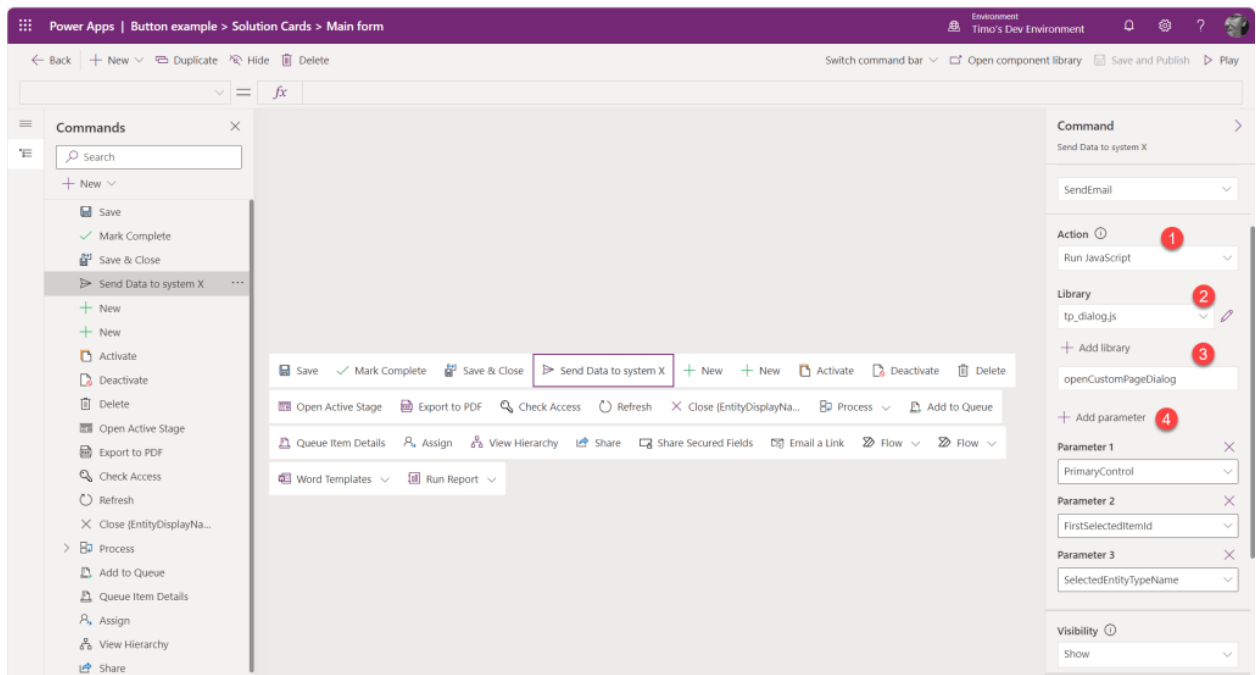
```
};  
var navigationOptions = {  
    target: 2,  
    position: 1,  
    height: {  
        value: 350,  
        unit: "px"  
    },  
    width: {  
        value: 350,  
        unit: "px"  
    },  
    title: "Send Data to system X"  
};  
Xrm.Navigation.navigateTo(pageInput, navigationOptions).then(  
    function () {  
        // Refresh the main form when the dialog is closed  
        primaryControl.data.refresh();  
    }  
)  
.catch (  
    function (error) {  
        // Handle error  
    }  
);  
}
```

Let's add JavaScript to the solution package as its own resource (web resource).

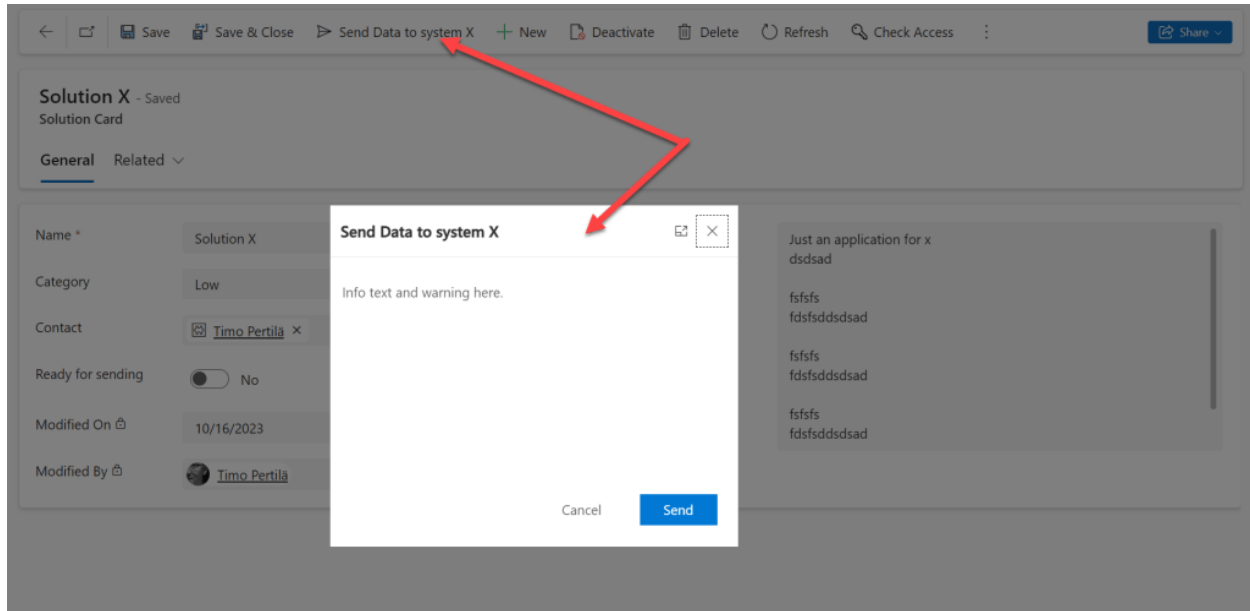


And go back to editing the command bar of the solution card. Let's change the action to Execute JavaScript (1). The web resource we just created is used as a library (2). The function (3) to be called is openCustomPageDialog found in JavaScript.

PrimaryControl, FirstSelectedItemId and SelectedEntityTypeName are passed as parameters (4) .



Now, when you press the button on the command bar, the dialog we built opens on the screen.



When the user presses “Send”, we forward the information and at the same time tell the user whether the transmission was successful.

Please note that custom pages do not work in my experience if mcas (Microsoft Defender for Cloud Apps) is in use in the organization.

Summary

This was again a perfect example of how the same problem can be solved in numerous different ways. All have their own pros and cons.

- And PCF controls were completely ignored (e.g. <https://github.com/MscrmTools/PCF-Controls#action-button>).
- [Integrate cloud flows with Power Apps - Power Automate | Microsoft Learn](#)
- [Different ways to trigger the flow from the model-driven app - Forward Forever](#)
- [Create a canvas app that can trigger a Power Automate flow - Power Apps | Microsoft Learn](#)
- [Start a flow in a canvas app - Power Apps | Microsoft Learn](#) – retired

Yes, you can absolutely replace the "email Excel attachment → user edits → emails back" pattern with either a Canvas App or a Model-Driven App, but I'd strongly lean toward Canvas App for your scenario.

Current Process

1 Power BI Dataset

- 2 ↓
- 3 Power Automate
- 4 ↓
- 5 Excel File w/ Formulas
- 6 ↓
- 7 Email to User
- 8 ↓
- 9 User Completes
- 10 ↓
- 11 Emails Back
- 12 ↓
- 13 Manual Processing

This process has a few challenges:

- Version control issues
- Email attachments everywhere
- Users can break formulas
- Difficult to track completion
- Hard to enforce data validation

Option 1 (Recommended): Canvas App

Instead of generating Excel files, write the Power BI dataset data into a table and let users update it through a Power App.

Architecture

- 1 Power BI Dataset
- 2 ↓
- 3 Power Automate

4 ↓

5 Dataverse Table

6 ↓

7 Canvas App

8 ↓

9 User Updates Data

10 ↓

11 Dataverse

12 ↓

13 Approval / Reporting Flow

How It Works

1. Create Dataverse Table

Example:

Column	Type
RequestID	Text
UserUPN	Text
SalesAmount	Currency
Forecast	Number
Comments	Multiline Text
Status	Choice
DueDate	Date

2. Load Data

Power Automate:

1 Refresh Power BI Dataset

2 ↓

3 Run DAX Query

4 ↓

5 For Each Record

6 ↓

7 Upsert Dataverse Record

3. Canvas App Security

Filter records by Entra ID:

```
1 Filter(  
2   ForecastTable,  
3   UserUPN = User().Email  
4 )
```

User automatically sees only their records.

No emails required.

4. Formula Replacement

Many Excel formulas can be recreated directly in Power Apps.

Example:

Excel:

```
1 =A1*B1
```

Canvas App:

```
1 txtQty.Value * txtPrice.Value
```

Or calculated columns in Dataverse.

5. Submit Changes


```
1 Patch(  
2   ForecastTable,  
3   ThisItem,  
4   {  
5     Forecast: Value(txtForecast.Text),  
6     Comments: txtComments.Text,  
7     Status: "Submitted"  
8   }  
9 )
```

Option 2: Model-Driven App

If the form is more data-entry oriented and less customized.

Architecture

```
1 Power BI Dataset  
2   ↓  
3 Dataverse  
4   ↓  
5 Model Driven App
```

Benefits:

- Faster to build
- Security built in
- Auditing
- Business Rules
- Approval workflows

Great when:

- Large user population

- Lots of fields
- Simple forms
- Manager approvals

Not as pretty as Canvas Apps.

Option 3: Keep Excel but Use App Launcher

If users absolutely need Excel formulas.

Flow

- 1 Power BI Dataset
- 2 ↓
- 3 Create Excel in SharePoint
- 4 ↓
- 5 Assign User
- 6 ↓
- 7 Canvas App displays link
- 8 ↓
- 9 User opens Excel Online
- 10 ↓
- 11 Submit

Instead of emailing files around, each user gets their own workbook stored in SharePoint.

This is a common "least disruption" upgrade.

Option 4: Power BI Writeback (My Favorite)

If this is truly a forecasting, planning, budgeting, or commentary process:

- 1 Power BI Dataset
- 2 ↓

3 Canvas App

4 ↓

5 Dataverse

6 ↓

7 Power BI Report

Embed the Canvas App directly inside Power BI.

Users:

1. Open report
2. See their numbers
3. Update forecast/comments
4. Save
5. Refresh report

No Excel at all.

If I Were Building This Today

I'd do:

1 Power BI Dataset

2 ↓

3 Power Automate

4 ↓

5 Dataverse

6 ↓

7 Canvas App

8 ↓

9 Manager Approval Flow

10 ↓

11 Power BI Dashboard

Then use:

- Entra ID security
- Dataverse tables
- Canvas App forms
- Power Automate approvals
- Power BI reporting

You'll eliminate attachment emails entirely and gain real-time tracking of who has/hasn't submitted.

For your AZ-204 and Power Platform journey, this is also the most scalable enterprise pattern and aligns closely with how Microsoft recommends replacing Excel collection processes with Power Apps + Dataverse.